Auto Tunning Hyperparameter with Optuna

14 May 2024 9:38

1. What is hyper parameter and what it does mean to the machine learning process.
   Sure, here are the key points about hyperparameters in machine learning:
   a. Definition of Hyperparameters:
      i. Hyperparameters are parameters that are explicitly defined by the user to control the learning process.
      ii. They are usually set before the learning process begins and are external to the model.
      iii. The prefix 'hyper' suggests that they are 'top-level' parameters that control the learning process and the model parameters that result from it.
   b. Role of Hyperparameters in Machine Learning:
      i. Hyperparameters control the learning process of the model, such as the learning rate, the number of neurons in a neural network, or the kernel size in a support vector machine.
      ii. They are often used to tune the performance of a model, and they can have a significant impact on the model's accuracy, generalization, and other metrics.
      iii. The optimal values of hyperparameters are usually determined either by the rule of thumb or by trial and error.
   c. Difference Between Hyperparameters and Model Parameters:
      i. Model parameters are configuration variables that are internal to the model, and a model learns them on its own.
      ii. For example, weights or coefficients of independent variables in the Linear regression model, or weights and biases of a neural network, are model parameters.
      iii. On the other hand, hyperparameters are those parameters that are explicitly defined by the user to control the learning process.
   d. Examples of Hyperparameters:
      i. The k in kNN or K-Nearest Neighbour algorithm.
      ii. Learning rate for training a neural network.
      iii. Train-test split ratio
      iv. .Batch Size.
      v. Number of Epochs.
      vi. Branches in Decision Tree.
      vii. Number of clusters in Clustering Algorithm[1].

2. Hyperparameter Tuning:
   a. Hyperparameter tuning is the process of selecting the optimal values for a machine learning model's hyperparameters.
   b. The goal of hyperparameter tuning is to find the values that lead to the best performance on a given task.
   c. In the context of machine learning, "the values that lead to the best performance on a given task" typically refer to the optimal values of model parameters and hyperparameters that result in the highest accuracy, precision, recall, or other relevant performance metrics for a specific task.
   d. In summary, hyperparameters are crucial in machine learning as they control the learning process of the model and can significantly impact the model's performance. The optimal values of hyperparameters are usually determined through a process called hyperparameter tuning.

3. Motivation behind the auto tunning of the hyperparameter
   a. Fair comparison with the baseline.
   b. Automatic Tunning, no more grad student descent.

c. Improve performance/training time.

4. Sampler
   a. N vs B/N Tradeoff

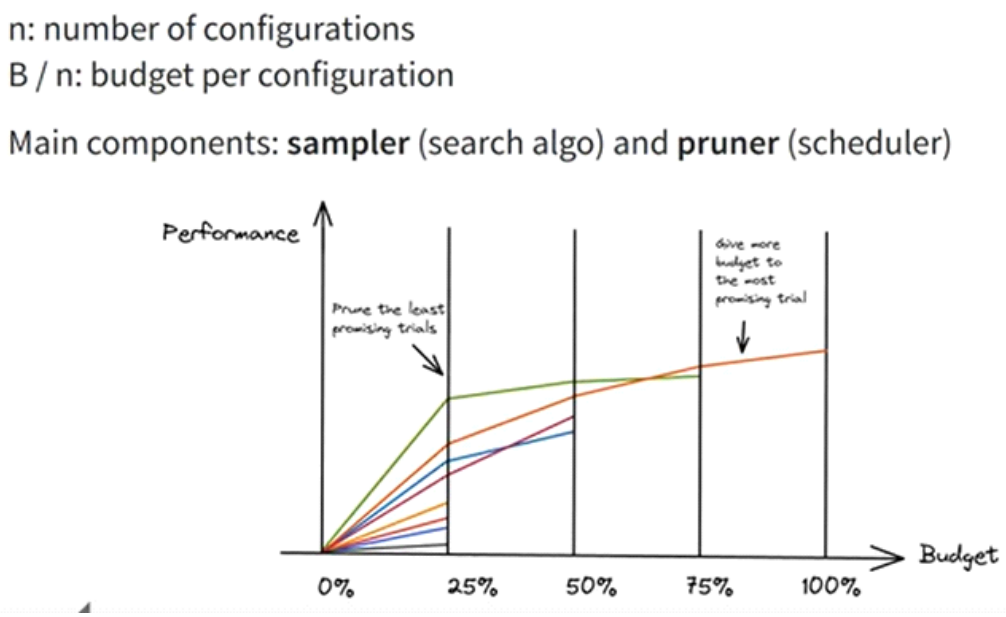
      The "N vs B/N" tradeoff in hyperparameter automatic optimization refers to the balance between the number of configurations (N) you try and the resources (B) you allocate to each configuration. Here's a more detailed explanation:

      i. N (Number of Configurations): This represents the number of different sets of hyperparameters (configurations) you want to try during the optimization process.
      ii. B (Budget): This is the total amount of computational resources you have available for the optimization process.
      iii. B/N (Budget per Configuration): This is the amount of computational resources you allocate to each configuration.

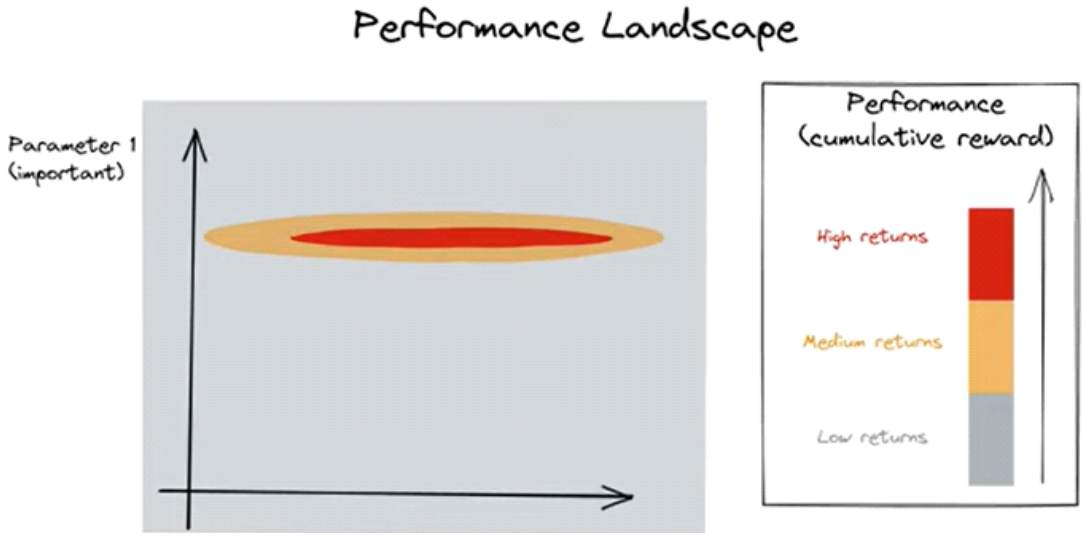
      The tradeoff is about how to best allocate your resources:

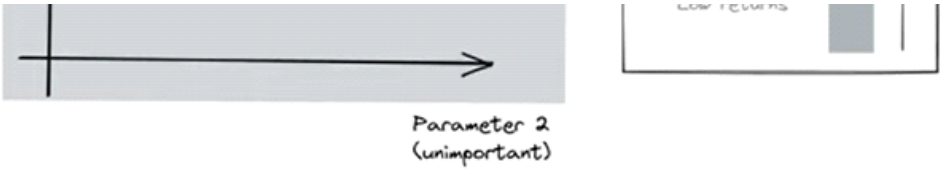
      i. If you try many configurations (large N), you can explore a wider range of possibilities, but you'll have fewer resources for each configuration (small B/N), which might lead to less accurate evaluations.
      ii. If you try fewer configurations (small N), you can allocate more resources to each configuration (large B/N), which might lead to more accurate evaluations, but you'll explore a narrower range of possibilities.

      The goal of hyperparameter optimization is to find the right balance between exploration (trying many different configurations) and exploitation (allocating enough resources to accurately evaluate each configuration). This balance is crucial for finding the best hyperparameters efficiently.



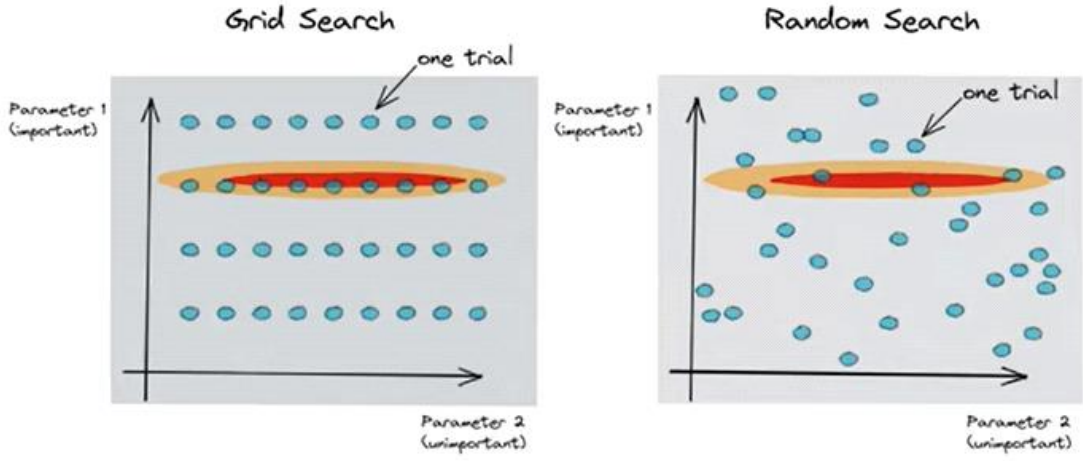
   b. Performance Landscape





Sure, here are some key points about performance landscapes in the context of automatic hyperparameter optimization in machine learning:

i. Definition of Performance Landscapes:
   1) In the context of hyperparameter optimization, a performance landscape is a representation of model performance across different hyperparameter configurations.
   2) Each point in the landscape corresponds to a specific set of hyperparameters, and its height (or depth) represents the performance of the model with those hyperparameters.
ii. Role of Performance Landscapes in Hyperparameter Optimization:
   1) Performance landscapes help us understand the structure of the optimization problem.
   2) They can reveal patterns such as the presence of local optima, plateaus, and ridges, which can inform the choice of optimization algorithm.
iii. Characteristics of Performance Landscapes:
   1) Performance landscapes can be highly complex, with many local optima and high dimensionality.
   2) The landscape can change dramatically depending on the dataset, the model, and even the metric used to evaluate performance.
iv. Exploratory Landscape Analysis (ELA):
   1) ELA is a set of techniques used to analyze performance landscapes.
   2) It can provide insights into the difficulty of the optimization problem and help identify suitable optimization algorithms.
v. Challenges and Future Directions:
   1) Understanding performance landscapes is a challenging task due to their complexity and high dimensionality.
   2) Future research could focus on developing more effective methods for landscape analysis and visualization.
vi. In summary, performance landscapes play a crucial role in hyperparameter optimization. They provide a way to visualize and understand the optimization problem, which can guide the selection of suitable optimization algorithms.
vii. Grid search vs random search in performance landscape



Sure, here's a comparison of Grid Search and Random Search in the context of hyperparameter optimization:

1) Grid Search:
    a) Grid Search is a traditional method where you define a grid of hyperparameters and train a model for every possible combination of these hyperparameters.
    b) It ensures a thorough exploration of the hyperparameter space but can be computationally expensive.
    c) The number of models to train grows exponentially as you increase the number of hyperparameters to tune, which can be very inefficient in terms of computing power and time.
    d) However, Grid Search guarantees the best subset of hyperparameters from the specified list.
2) Random Search:
    a) In Random Search, you define a statistical distribution for each hyperparameter from which values are sampled.
    b) Unlike Grid Search, in which every possible combination is evaluated, in Random Search, you can specify to train only a fixed number of models and terminate the tuning algorithm post that.
    c) Random Search offers a more efficient alternative, especially for large search spaces.
    d) It may not always provide the most optimal results but is usually faster and more efficient, particularly in scenarios where the dimensionality of the hyperparameter space is high.

In the context of landscape analysis, both Grid Search and Random Search provide different "paths" through the hyperparameter space. Grid Search systematically explores the landscape, while Random Search jumps around randomly. The choice between the two often depends on the specific problem and the computational resources available.

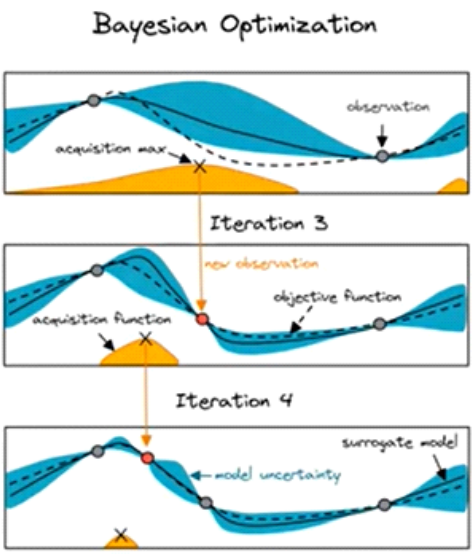
c. Bayesian Optimization

Bayesian Optimization is an optimization method that uses probabilistic models to efficiently find a model's hyperparameters. Here's a more detailed explanation:

i. Concept of Bayesian Optimization:
    1) Bayesian Optimization is a sequential design strategy for global optimization of black-box functions.
    2) It works by constructing a probabilistic model of the objective function and using it to select the most promising hyperparameters to evaluate in the true objective function.
    3) The goal is to find the optimal combination of hyperparameters.
ii. Role in Hyperparameter Optimization:
    1) Bayesian Optimization is used for hyperparameter tuning in machine learning.
    2) It helps reduce the time required to obtain an optimal parameter set and improves the performance of test set generalization tasks.
    3) It considers the previously seen hyperparameter combinations when determining the next set of hyperparameters to evaluate.
iii. Advantages of Bayesian Optimization:
    1) Bayesian Optimization is particularly useful when the cost of function evaluations is high.
    2) It can handle noise in function evaluations and does not require derivatives.
    3) It can also handle constraints and optimize over both continuous and discrete variables.

In summary, Bayesian Optimization is a powerful technique for hyperparameter optimization in machine learning. It uses probabilistic models to efficiently explore the hyperparameter space and find the optimal set of hyperparameters.

- Gaussian Process (GP)
- Tree of Parzen Estimators (TPE)



5. Scheduler
   a. Scheduler definition
      In the context of automatic hyperparameter optimization, a scheduler is a component that manages the allocation of computational resources to different trials[12]. Here's a more detailed explanation:
      i. Definition of Scheduler:
         1) A scheduler is responsible for deciding which trials to run, when to run them, and how to allocate resources to them.
         2) It helps manage the computational budget and improve the efficiency of the optimization process.
      ii. Role in Hyperparameter Optimization:
         1) The scheduler plays a crucial role in managing the exploration and exploitation trade-off in hyperparameter optimization.
         2) It can decide to allocate more resources to promising trials (exploitation) or to try out new hyperparameter configurations (exploration).
      iii. Types of Schedulers:
         1) There are different types of schedulers, each with its own strategy for managing trials.
         2) For example, the ASHA scheduler uses an early-stopping strategy to terminate unpromising trials, thereby freeing up resources for more promising ones.
      iv. Schedulers vs Search Algorithms:
         While the search algorithm governs how the hyperparameter space is sampled and optimized, the scheduler improves the overall efficiency of the hyperparameter optimization by terminating unpromising trials early.

      In summary, a scheduler in hyperparameter optimization is a component that manages the allocation of computational resources to different trials, helping to balance the trade-off between exploration and exploitation.

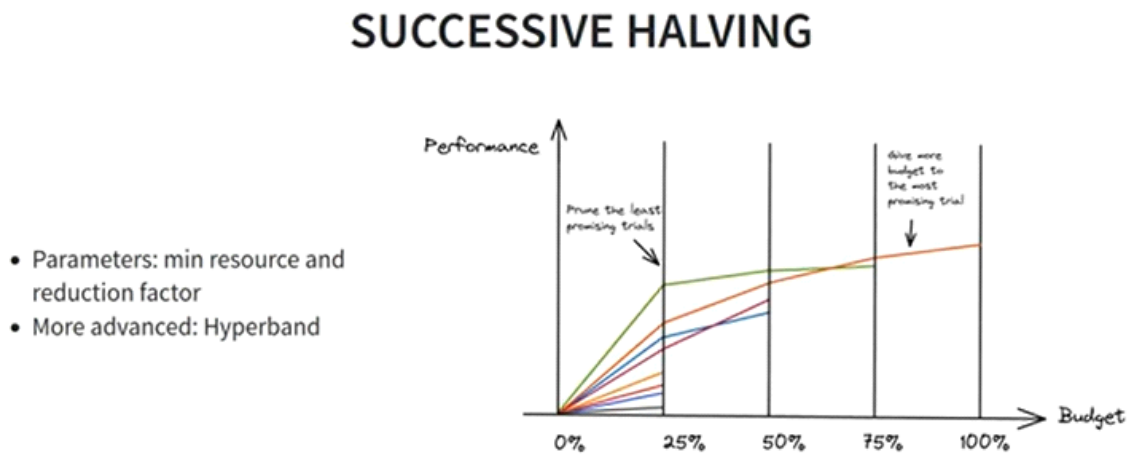
   b. Median Pruner
      The Median Pruner is a component used in the process of automatic hyperparameter optimization[3]. Here's a more detailed explanation:
      i. Definition of Median Pruner:
         1) The Median Pruner is a pruning strategy used during the optimization proces.
         2) It uses the median stopping rule, which means it stops a trial if the trial's best intermediate result is worse than the median of intermediate results of previous trials at the same step.
      ii. Role in Hyperparameter Optimization:

1) The Median Pruner helps to efficiently allocate computational resources during the optimization process.
2) By stopping unpromising trials early, it allows more resources to be allocated to more promising trials.
3) This can speed up the optimization process and improve the chances of finding the optimal set of hyperparameters.

iii. Usage in Schedulers:
1) In the context of schedulers, the Median Pruner can be used to decide when to stop a trial based on its performance relative to other trials.
2) This can be particularly useful in scenarios where computational resources are limited.

In summary, the Median Pruner is a tool used in automatic hyperparameter optimization to efficiently allocate computational resources by stopping unpromising trials early.

c. Successive Halving



Successive Halving is a method used in hyperparameter optimization, particularly in the context of scheduling. Here's a more detailed explanation:

i. Concept of Successive Halving:
1) Successive Halving is an iterative method for hyperparameter optimization.
2) In each iteration, it evaluates a number of configurations and only keeps the best half for the next iteration.
3) The number of configurations decreases exponentially from one iteration to the next, while the number of training cases (or the amount of resources) increases exponentially.

ii. Role in Hyperparameter Optimization:
1) Successive Halving is used to efficiently allocate computational resources during the optimization process.
2) By discarding unpromising configurations early, it allows more resources to be allocated to more promising ones.
3) This can speed up the optimization process and improve the chances of finding the optimal set of hyperparameters.

iii. Usage in Schedulers:
1) In the context of schedulers, Successive Halving can be used to decide when to stop a trial based on its performance relative to other trials.
2) This can be particularly useful in scenarios where computational resources are limited.

In summary, Successive Halving is a method used in automatic hyperparameter optimization to efficiently allocate computational resources by stopping unpromising trials early.

6. Optuna: Optuna is a library that helps you to automate the search. In this Unit, we'll study a little bit of the theory behind automatic hyperparameter tuning. We'll first try to optimize the

parameters of the DQN studied in the last unit manually. We'll then learn how to automate the search using Optuna.

a. What is optuna and its capabilities.
   Optuna is an open-source software framework for automatic hyperparameter optimization[1,2]. Here are some key features and details about Optuna:
   i. Hyperparameter Optimization: Optuna automates the process of finding the best hyperparameters for machine learning models.
   ii. Define-by-Run API: Optuna features a user-friendly API that allows you to define the hyperparameter search space dynamically.
   iii. Efficient Optimization Algorithms: Optuna uses state-of-the-art algorithms for efficiently sampling hyperparameters and pruning unpromising trials.
   iv. Parallelization: Optuna supports easy parallelization, which means you can scale your optimization process across multiple threads or processes without modifying your code.
   v. Visualization: Optuna provides a variety of plotting functions that allow you to visually inspect the optimization history and hyperparameter importances.
   vi. Integration with Machine Learning Libraries: Optuna is framework agnostic, meaning you can use it with any machine learning or deep learning library.
   vii. In summary, Optuna is a powerful tool for automating and accelerating the hyperparameter optimization process in machine learning.

b. Hyper Parameter Tunning Step by Using Optuna:
   i. Define the Search Space:
      1) The search space includes all the hyperparameters that you want to tune for your model.
      2) In Optuna, you define the search space within the objective function using the suggest methods.
      3) For example, if you want to tune the learning_rate and gamma parameters of a DQN agent, you might define your search space as follows:

```
defobjective(trial):
   learning_rate = trial.suggest_loguniform('learning_rate', 1e-5, 1e-1)
   gamma = trial.suggest_uniform('gamma', 0.8, 0.9999)
```

   ii. Define the Objective Function:
      1) The objective function is what you want to optimize.
      2) In machine learning, this is typically the validation error of your model.
      3) You define the objective function in Optuna as a Python function that takes a trial object and returns a numerical value.
      4) For example, if you're training a DQN agent, your objective function might look like this:

```
fromstable_baselines3 importDQN
defobjective(trial):
   learning_rate = trial.suggest_loguniform('learning_rate', 1e-5, 1e-1)
   gamma = trial.suggest_uniform('gamma', 0.8, 0.9999)
   model = DQN('MlpPolicy', 'CartPole-v1', gamma=gamma,
learning_rate=learning_rate)
   model.learn(total_timesteps=10000)
   rewards = evaluate_model(model)  # This is a function you'd need to
definereturnrewards
```

   iii. Choose Sampler and Scheduler:
      1) The sampler determines how to sample the search space.
      2) The scheduler decides when to stop trials.
      3) Optuna provides several options for both, such as the TPE sampler and the Median Pruner.
      4) You can set the sampler and pruner when you create the study object:

```
fromoptuna.samplers importTPESampler
fromoptuna.pruners importMedianPruner
sampler = TPESampler()
```

```
pruner = MedianPruner()
study = optuna.create_study(sampler=sampler, pruner=pruner)
study.optimize(objective, n_trials=100)
```

In summary, hyperparameter tuning in Optuna involves defining the search space and the objective function, and choosing a sampler and a scheduler. These steps allow Optuna to efficiently explore the hyperparameter space and find the optimal set of hyperparameters.